UNIVERSIDAD POPULAR DEL CESAR

Ingeniería de Sistemas e Informática

GUÍA DEL TALLER

Refactorización a Código Limpio

Sistema de Facturación — Restaurante El Buen Sabor

Programación de Computadores III · SS462 · Cuarto Semestre

Docente: Ing. Alfredo David Bautista Romero

1. ¿Qué van a encontrar?

El proyecto RestauranteBuenSabor es un sistema de facturación para un restaurante que funciona correctamente, pero está escrito con el máximo de malas prácticas posibles. Su misión es refactorizarlo hasta convertirlo en código limpio, sin alterar el comportamiento del programa.

> ⚠ *El programa produce resultados correctos. El problema no es que falle — el problema es que es imposible de mantener, entender y modificar. Eso es exactamente lo que pasa en la industria real.*

1.1 Estructura de archivos entregada

```
RestauranteBuenSabor/
├── src/
│   └── restaurante/
│       ├── Main.java        — punto de entrada y menú interactivo
│       ├── Datos.java       — almacén global de datos del sistema
│       ├── Proceso.java     — lógica de cálculo de la factura
│       ├── Utilidades.java  — métodos auxiliares
│       └── Imprimir.java    — todo lo que se imprime en pantalla
└── README.md                — instrucciones para compilar y ejecutar
```

2. ¿Qué hace el sistema?

El programa simula la operación de caja de un restaurante. Al ejecutarlo aparece un menú con 6 opciones que se repite en bucle hasta que el usuario elige salir:

```
========================================
    RESTAURANTE EL BUEN SABOR
    Calle 15 #8-32, Valledupar
    NIT: 900.123.456-7
========================================
1. Ver carta
2. Agregar producto al pedido
3. Ver pedido actual
4. Generar factura
5. Nueva mesa
0. Salir
========================================
Seleccione una opcion:
```

2.1 Carta del restaurante (Opción 1)

#	Producto	Precio
1	Bandeja Paisa	$32.000
2	Sancocho de Gallina	$28.000
3	Arepa con Huevo	$8.000

4	Jugo Natural	$7.000
5	Gaseosa	$4.500
6	Cerveza Poker	$6.000
7	Agua Panela	$3.500
8	Arroz con Pollo	$25.000

2.2 Reglas de negocio (Opción 4 — Generar factura)

- Subtotal: suma de precio × cantidad para cada producto pedido.
- Descuento 5%: se aplica si hay más de 3 productos diferentes en el pedido.
- IVA 19%: se calcula sobre el subtotal después del descuento.
- Propina 10%: se aplica si el subtotal (antes del IVA) supera $50.000.
- Total: subtotal con descuento + IVA + propina.

Ejemplo de factura generada (Bandeja Paisa x2 + Jugo Natural x2):

```
FACTURA No. 001
----------------------------------------
Bandeja Paisa        x2      $64.000
Jugo Natural         x2      $14.000
----------------------------------------
Subtotal:                   $78.000
IVA (19%):                  $14.820
Propina (10%):              $ 9.282
----------------------------------------
TOTAL:                      $102.102
```

3. Catálogo de Malas Prácticas

A continuación, se documenta cada mala práctica presente en cada archivo, con su ubicación y descripción. Usen esto como lista de verificación durante la refactorización.

3.1 Datos.java — El God Object

Almacena todos los datos del sistema en variables estáticas públicas. Cualquier clase puede leer y modificar cualquier dato directamente, sin control ni encapsulamiento.

#	Problema	Dónde	Descripción
1	Arrays paralelos	String[] nom, double[] p, int[] cant	Tres arrays separados para nombre, precio y cantidad. Para agregar un producto hay que modificar los tres en la misma posición.
2	Todo public static	Todas las variables	Cualquier clase puede leer y modificar cualquier dato sin control. Sin encapsulamiento.
3	Nombre hardcodeado	String x = "El Buen Sabor"	El nombre del restaurante está en una variable llamada x (poco descriptiva) y escrita directamente.
4	Variable tmp sin propósito	public static String tmp	Se usa como variable temporal global, pero su estado es impredecible y se modifica desde varios lugares.

5	Sin constructor ni métodos		La clase es solo un contenedor de datos globales, no tiene comportamiento ni protege su estado.
6	Nombres de 1-3 letras	ms, est, tot, nf, x, tmp, nom, p, cant	Imposible saber qué representan sin leer todo el código.

3.2 Proceso.java — El método que hace todo

Contiene la lógica de cálculo del total de la factura en un único método de más de 80 líneas.

#	Problema	Dónde	Descripción
7	hacerTodo() con múltiples responsabilidades	Líneas 3–37	Calcula subtotal, cuenta items, aplica descuento, IVA, propina, actualiza estado y retorna total.
8	IVA duplicado 3 veces	Líneas 15, 21, 29	iva = aux * 0.19 aparece copiado en cada rama del if-else.
9	Magic numbers sin nombre	0.19, 0.10, 0.05, 50000, 3	No se entiende qué significan esos números sin leer las reglas de negocio.
10	Condicionales anidados 4 niveles	Líneas 11–35	if(cont>3){ if(sub>0){ if(aux>50000){...}else{...} } }else{ if(sub>50000){...} }
11	Código comentado 'versión anterior'	Líneas 23–26	Código antiguo comentado que confunde sobre qué lógica está activa.
12	procesar() con 7 parámetros	Línea 38	procesar(double a, double b, double c, double d, double e, int f, boolean g) – nombres sin sentido, duplica lógica.
13	Variable aux innecesaria	Línea 4	Guarda un valor intermedio que se podría calcular directamente.
14	Comentarios que repiten el código	Líneas 7, 20, 30	// multiplica precio por cantidad encima de sub=sub+Datos.p[i]*Datos.cant[i] – el código ya lo dice.
15	Comentarios de cierre inútiles	// fin while, // fin if	No aportan información que la llave de cierre no comunique.

3.3 Utilidades.java — Nombres engañosos

Contiene métodos auxiliares: calcular, validar y reiniciar el pedido.

#	Problema	Dónde	Descripción
16	calcular() con 7 parámetros	Línea 3	calcular(double pr, double cn, double dc, double iv, double pp, int ni, boolean ap) – duplica lógica de Proceso.
17	validar() con efecto secundario oculto	Líneas 19–20	El método se llama validar() pero también modifica Datos.tot y Datos.tmp. El nombre promete solo lectura.
18	Nombre del restaurante hardcodeado (3ra vez)	Línea 11	Dentro de calcular() imprime "RESTAURANTE EL BUEN SABOR - calculo aplicado" – un método de cálculo no debería imprimir.

19	Bloque de ~20 líneas de código comentado	Líneas 23–38	Versión anterior completa del método abandonada dentro de reiniciar(). Contamina visualmente.
20	Variables tmp y aux2 redundantes	Líneas 4–12	tmp copia el IVA, aux2 copia el resultado final. Ambas innecesarias

3.4 Imprimir.java — Código copiado y pegado

Contiene todos los métodos de impresión en consola.

#	Problema	Dónde	Descripción
21	imprimirFacturaCompleta() con 3 responsabilidades	Líneas 25–69	Calcula el total (duplicando Proceso), imprime la factura y actualiza contador de facturas y estado de mesa.
22	Cálculo del total duplicado	Líneas 26–41 y 71–86	El mismo cálculo aparece copiado en imprimirFacturaCompleta() e imprimirFacturaResumen().
23	80% del código idéntico entre métodos	Líneas 25–101	La única diferencia es que el resumen omite el detalle de items. Todo lo demás copiado.
24	Nombre del restaurante hardcodeado (4ta vez)	Líneas 44–46 y 88–90	"RESTAURANTE EL BUEN SABOR", "Calle 15 #8-32" y "NIT: 900.123.456-7" escritos literalmente.
25	Formato printf repetido sin constante	Líneas 10, 19, 53, 56, 95	"%-20s x%-6d $%,.0f%n" repetido sin extraer como constante.

3.5 Main.java — El caos central

Punto de entrada del programa. El método main mezcla presentación, validación y lógica de negocio en más de 100 líneas.

#	Problema	Dónde	Descripción
26	Método main de 100+ líneas	main()	Todo el flujo del menú, validaciones, llamadas a negocio y presentación en un único método.
27	Dos Scanner para el mismo System.in	main()	Scanner sc y Scanner sc2 apuntan al mismo flujo de entrada – puede causar comportamiento impredecible.
28	Variables sin uso	boolean continuar	Declarada pero nunca se lee para tomar decisiones reales.
29	Lógica de negocio en el main	if-else anidados	Validaciones, cálculos parciales y actualizaciones de estado mezclados con el control del menú.
30	Nombre del restaurante hardcodeado (5ta vez)	System.out.println	Aparece directamente en un println del main junto con dirección y NIT.

3.6 Problemas de formato visual (todos los archivos)

#	Problema	Dónde	Descripción
31	Sin indentación	Todos los archivos	Todo el código pegado al margen izquierdo. Imposible ver jerarquía.
32	Llaves inconsistentes	Todos los archivos	A veces la llave de apertura va en misma línea, a veces en la siguiente.
33	Llaves de cierre apiladas	Proceso, Main	}}} al final de bloques anidados sin líneas en blanco.
34	Múltiples statements por línea	Todos los archivos	int a=0;boolean b=true;int c=0; en la misma línea.

Total: 34 malas prácticas identificadas y documentadas en el proyecto.

Referencia principal: Robert C. Martin — Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2009.

Fuente complementaria: Shahan Chowdhury — The Clean Code Handbook. FreeCodeCamp, Enero 2025.